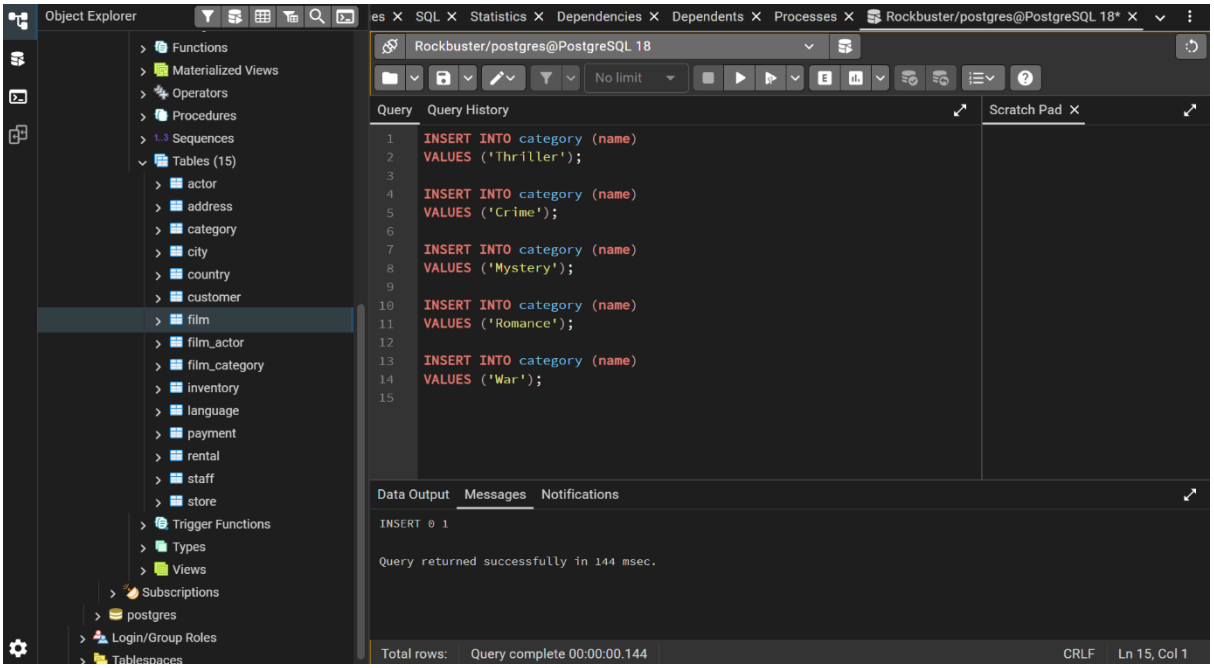
SQL for Data Analysts

Find out what film genres already exist in the category table:

- *Open pgAdmin 4, click the Rockbuster database, and open the Query Tool.*
- *Write a SELECT command to find out what film genres exist in the category table.*

1	**"Action"**
2	**"Animation"**
3	**"Children"**
4	**"Classics"**
5	**"Comedy"**
6	**"Documentary"**
7	**"Drama"**
8	**"Family"**
9	**"Foreign"**
10	**"Games"**
11	**"Horror"**
12	**"Music"**
13	**"New"**
14	**"Sci-Fi"**
15	**"Sports"**
16	**"Travel"**

Write an INSERT statement to add the following genres to the category table: Thriller, Crime, Mystery, Romance, and War



INSERT INTO category (name)

VALUES ('Thriller');

INSERT INTO category (name)

VALUES ('Crime');

INSERT INTO category (name)

VALUES ('Mystery');

INSERT INTO category (name)

VALUES ('Romance');

INSERT INTO category (name)

VALUES ('War');

The CREATE statement below shows the constraints on the category table. Write a short paragraph explaining the various constraints that have been applied to the columns. What do these constraints do exactly? Why are they important?

```
CREATE TABLE category
(
  category_id integer NOT NULL DEFAULT nextval('category_category_id_seq'::regclass),
  name text COLLATE pg_catalog."default" NOT NULL,
  last_update timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT category_pkey PRIMARY KEY (category_id)
);
```

Explanation:

- category_id is the primary key, ensuring each genre has a unique identifier. The DEFAULT nextval(...) automatically generates new IDs, preventing duplicates.
- name is marked **NOT NULL**, ensuring that every category must have a genre name.
- last_update automatically records the timestamp of when the row was created or updated.
- The primary key constraint enforces data integrity and allows efficient joins with other tables, such as film_category.

These constraints are important because they maintain data accuracy, consistency, and reliability across the database.

The genre for the movie African Egg needs to be updated to thriller. Work through the steps below to make this change. Write the SELECT statement to find the film_id for the movie African Egg.

```
SELECT
  film_id,
  title
```

```
        film_category
FROM film
WHERE title = 'African Egg';
```

Once you have the film_ID and category_ID, write an UPDATE command to change the category in the film_category table (not the category table). Copy-paste this command into your answers document.

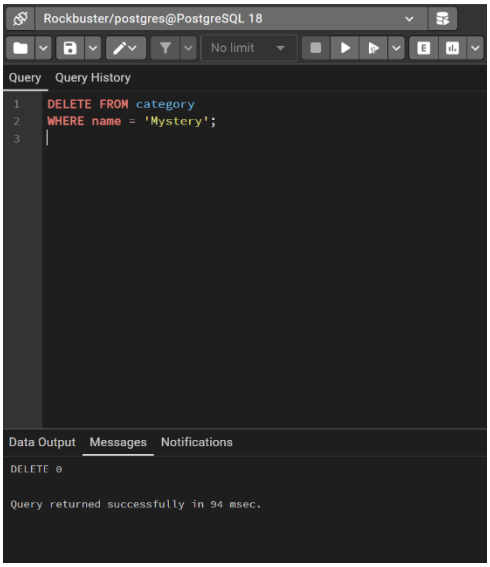
```
SELECT
  category_id,
  name
FROM category
WHERE name = 'Thriller';

UPDATE film_category
SET category_id = (
  SELECT category_id
  FROM category
  WHERE name = 'Thriller'
)
WHERE film_id = (
  SELECT film_id
  FROM film
  WHERE title = 'African Egg'
);
```

Since there aren't many movies in the mystery category, you and your manager decide to remove it from the category table. Write a DELETE command to do so and copy-paste it into your answers document.

DELETE FROM category

WHERE name = 'Mystery';



Based on what you've learned so far, think about what it would be like to complete steps 1 to 4 with Excel instead of SQL. Are there any pros and cons to using SQL? Write a paragraph explaining your answer.

If I had to complete steps 1 to 4 using Excel instead of SQL, it would be much more difficult and less reliable. In Excel, I would need to manually filter, edit, and keep track of film genres, which increases the risk of mistakes such as duplicate categories or inconsistent updates across different sheets. Excel also doesn't enforce rules like primary keys or relationships between tables, so it's easy to accidentally break the structure of the data. Using SQL is more efficient because it allows me to directly query, insert, update, and delete data while maintaining data integrity. SQL also ensures that relationships between tables, such as films and categories, remain consistent. While Excel is useful for quick analysis and small datasets, SQL is better suited for managing structured databases like Rockbuster where accuracy and scalability are important.